האוניברסיטה העברית

ביה"ס להנדסה ולמדעי המחשב

בחינה בקורס מבוא למדעי המחשב 67101

מועד ב' – סמסטר א' – תשע"ה – 2014-2015 **תאריך:** 31.3.2015

ד"ר אביב זהר, פרופ' יובל רבני. **משך הבחינה:** שלוש שעות

הנחיות:

1. הבחינה עם **חומר סגור**. אין להביא חומר עזר כלשהו לבחינה (כולל ספרים, מחברות, דפים ומכשירים אלקטרוניים). **יש להיעזר רק בדף המצורף לבחינה, ובו רשימת פונקציות ומתודות שניתן להשתמש בהן.**
2. יש לענות על **3 מתוך 4** השאלות הבאות. משקל כל השאלות זהה (33 נק'), נקודה אחת נוספת מוענקת אוטומטית, כך שהציון המקסימלי הוא 100.
3. **יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור. אין לענות על גבי טופס הבחינה - הוא לא ייסרק בסיום הבחינה ולא יבדק.**
4. יש לציין את מס' תעודת הזהות ומספר המחברת על **כל** מחברת.
5. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
6. **מותר להשתמש בכל הפונקציות והמתודות הכתובות בדף המצורף לבחינה, וכן בכל הפונקציות שהן built-in בפייתון, בתנאי שהן לא פותרות ישירות את השאלה.**
7. בפייתון ההזחה (אינדנטציה) היא קריטית להבנת הקוד – הקפידו על הזחה ברורה ונכונה!
8. יש למחוק טיוטות באופן ברור. הבדיקה לא תתייחס לטיוטות כלל.
9. **יש להשאיר שוליים.**
10. **אסור לכתוב בעט אדום.**
11. הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
12. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית). אין צורך לכתוב docstring.
13. פתרון מיטבי לשאלה יחשב פתרון **פשוט ויעיל ככל האפשר**. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
14. אין להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להשתמש ברשימה בת שלשה תאים פעם אחת).
15. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. ניתן להניח כי מבני הנתונים מכילים רק ערכים תקינים ואינם **None**, אלא אם מובהר בשאלה אחרת.
16. בפתרון השאלה אתם רשאים לבצע שימוש בפונקציות ומתודות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בפונקציות או מתודות משאלות אחרות).

בהצלחה!

שאלה 1 (33 נקודות)

יש לענות על הסעיפים הבאים על פי הקוד הנתון:

```
def f(x,y):
   return (x+y) % 2

def g(x):
   if len(x) == 1:
       return x[0]
   else:
       return x[0] + g(x[1:])
```

א. (3 נקודות)

```
print(g([0,1,0,1,0]))
print(g([1,1,1,1,0]))
```

מה יודפס?

ב. (1 נקודה)

הסבירו במילים מה מחשבת הפונקציה g.

```
def h(x):
   if len(x) == 1:
       return x[0]
   else:
       return f(x[0],h(x[1:]))
```

ג. (4 נקודות)

```
print(h([0,1,0,1,0]))
print(h([1,1,1,1,0]))
```

מה יודפס?

```
def foo(x,y):
   return g([f(x[i],y[i]) for i in range(len(y))])
```

ד. (3 נקודות)

```
print(foo([0,0,1,1,1,0,0], [1,1,1,1,1]))
```

מה יודפס?

ה. (2 נקודות)

הסבירו במילים מה מחשבת הפונקציה foo (מספיק להסביר מה יחושב עבור ארגומנטים שהם רשימות המכילות 1,0 בלבד).

```
def bar(x,y,z):
    if len(x) < len(y):
        return False
    elif foo(x,y) <= z:
        return True
    else:
        return bar(x[1:],y,z)
```

ו. (6 נקודות)

```
print(bar([0,0,1,1,1,0,0,0,1,1,0,0,1,0,0,0], [1,1,1,1,1], 1))
print(bar([1,0,1,0,0,0,0,0,1,1,0,0,1,0,0,0], [1,1,1,1,1], 2))
```

מה יודפס?

ז. (4 נקודות)

רשמו את הפרמטרים (x,y,z) של הקריאה הרקורסיבית האחרונה (הקריאה העמוקה ביותר ברקורסיה) ל-bar בשורה הראשונה שבסעיף ו.

ח. (4 נקודות)

שנו את הקוד של bar כך שהפונקציה תעשה אותו הדבר אבל ללא רקורסיה.

ט. (3 נקודות)

ציינו לפחות שיפור אחד בסיבוכיות הזמן או המקום שהמימוש בסעיף הקודם נותן על פני המימוש המקורי (מספיק לציין את סיבוכיות המקום או הזמן בסעיפים הנ"ל, ולנמק במילים ללא הוכחה מתימטית של הסיבוכיות)

```
def foo_foo(x,y,z):
    if bar(x,y,z):
        for i in range(len(x)):
            if len(x) >= len(y)+i and foo(x[i:],y) <= z:
                yield i
```

י. (3 נקודות)

```
for i in foo_foo([0,1,0,1,0,0,1,1,1,1,0,1,0,1,1,1,1],[1,1,1,1,1],1):
    print(i)
```

מה יודפס?

שאלה 2 (33 נקודות)

נתונה המחלקה Node המייצגת איבר ברשימה מקושרת.

```
class Node(object):
   def __init__(self, data, next=None):
       self.data = data
       self.next = next
```

נאמר שרשימה היא **ללא מעגל** אם קיים בה איבר אחרון המצביע ל- None. אחרת, האיבר האחרון בה מצביע לאחד מאיברי הרשימה הקודמים (לא בהכרח לאיבר הראשון ברשימה) וברשימה יש מעגל. רשימה ריקה היא ללא מעגל.

נתונה הפונקציה has_cycle הבודקת אם ברשימה מקושרת יש מעגל:

```
def has_cycle(head):
   past = []
   cur = head
   while cur is not None:
       if cur in past:
           return True
       past.append(cur)
       cur = cur.next
   return False
```

א. (4 נקודות)
מה סיבוכיות הזמן והמקום של הפונקציה has_cycle?

סעיפי השאלה הבאים יכוונו אתכם לפתרון בזמן ליניארי של בעייה זו (כלומר בסיבוכיות O(n)), ובמקום קבוע.
בכל שלב ניתן לעשות שימוש ברעיונות מסעיפים קודמים אך לא בהכרח יעיל לקרוא ישירות לפונקציות שנכתבו בסעיפים קודמים.

ב. (5 נקודות)
ממשו את הפונקציה node_i המקבלת כארגומנט רשימה מקושרת (נתונה כמצביע לאיבר הראשון ברשימה) ואינדקס ומחזירה מצביע לאיבר ה i ברשימה. שימו לב כי ראש הרשימה מוגדר להיות האיבר עם אינדקס אפס ברשימה (i=0). אם הרשימה ריקה או שהאינדקס i מחוץ לטווח האינדקסים ברשימה, עליכם להחזיר None.

```
def node_i(head,i):
```

ג. (9 נקודות)
ממשו את הפונקציה `cycle_length` המקבלת כארגומנט רשימה מקושרת, ומחזירה את האינדקס של המופע השני של האיבר בראש הרשימה בתוך הרשימה. ניתן להניח כי ברשימה המתקבלת אכן יש מקום כזה (כלומר ברשימה הנתונה האיבר האחרון מצביע חזרה לראשון). הביטוי
`cycle_length(head)>0 and node_i(head,cycle_length(head)) is head`
צריך לתת ערך True. חתימת הפונקציה `cycle_length` היא:

```
def cycle_length(head):
```

ד. (10 נקודות)
שימו לב, בסעיף זה יש למצוא פתרון הרץ בזמן ריצה ליניארי ובמקום קבוע.
ממשו את הפונקציה `find_loop` המקבלת כארגומנט רשימה מקושרת ומחזירה את הערך הקטן ביותר i עבורו האיבר ה-i ברשימה זהה לאיבר ה 2*i+1 ברשימה. בפרט, ערך הביטוי
`node_i(head,i) is node_i(head,2*i+1)`
הוא True. ניתן להניח כי יש ברשימה מיקום כזה.

```
def find_loop(head):
```

ה. (5 נקודות)
כתבו מחדש את הפונקציה `has_cycle` מסעיף א', כך שתרוץ בזמן ליניארי ובמקום קבוע.

רמז: חישבו על ההנחה של סעיף ד'. האם היא מתקיימת כשאין מעגל?

שאלה 3 (33 נקודות)

נתון גנרטור שהאיטרטור שהוא מייצר מחזיר את כל תתי הרשימות של רשימת הקלט lst לפי סדר.

```
def subsets(lst):
    if len(lst)==0:
        yield []
    else:
        for tail in subsets(lst[1:]):
            yield tail
            yield [lst[0]]+tail
```

א: (4 נקודות)
מה יודפס? (הקפידו על סדר ההדפסה הנכון).

```
print(list(subsets([1,5,3])))
```

ב: (7 נקודות)
נתונה הפונקציה filter (שהיא built-in בפייתון):

```
filter(fun1, iter1)
```

המחזירה איטרטור שעובר על כל אברי iter1 עבורם הפונקציה fun1 מחזירה ערך אמת.
ממשו את הפונקציה הבאה:

```
filtered_subsets(lst,max_sum)
```

הפונקציה תייצר איטרטור המחזיר את כל תתי הרשימות של lst שסכום איבריהם קטן או שווה ל-max_sum. יש להשתמש בפונקציה filter הנתונה מעלה.

לדוגמא, הרשימה הבאה:

```
list(filtered_subsets([1,5,3],4))
```

תכיל כאחד האיברים שלה את הרשימה `[1,3]` אך לא את הרשימה `[1,5]` שסכום אבריה גדול מדי.

ג: (10 נקודות)
נניח שכל האיברים בקלט לגנרטור שלנו חיוביים. ממשו את הגנרטור

```
bounded_subsets(lst,max_sum)
```

הגנרטור הזה אמור לעשות בדיוק אותו הדבר שעושה הגנרטור בסעיף ב', אבל הפעם אין להשתמש ב-filter, אלא יש לשנות את הגנרטור subsets. האיטרטור המתקבל צריך "לחתוך" את ההרחבה של תתי רשימות שכבר עוברות את הסכום המותר. שימו לב: הפתרון בסעיף זה צריך להיות יעיל יותר מהפתרון לסעיף הקודם: עליו לעבור על פחות תתי רשימות במידת האפשר.

ד: (4 נקודות)
בסעיף ג' נוספה ההנחה שכל האיברים חיוביים, (אך לא בסעיף ב'). הראו מדוע. כלומר, הראו דוגמא לרשימה של מספרים בה הקוד של סעיף ב' יעבוד, ואילו הקוד של סעיף ג' יוציא פלט שגוי (הראו גם את הדוגמא וגם את הפלט השגוי).

ה: (8 נקודות)

שנו את הפונקציה `bounded_subsets` אותה ממשתם בסעיף ג' כדי ליצור פונקצייה `doubly_bounded_subsets(lst,min_sum,max_sum)`

המחזירה את כל תתי הרשימות שסכומן לפחות min_sum ולכל היותר max_sum.

גם בסעיף זה הניחו שכל אברי הרשימה `lst` הם חיוביים. שוב, האיטרטור המתקבל צריך להיות יעיל ו"לחתוך" את ההרחבה של תתי רשימות בשלב מוקדם אם הן בוודאי לא יהיו בין המינימום למקסימום המותרים.

שאלה 4 (33 נקודות)

נתונה המחלקה הבאה שמייצגת לקוח במכירה פומבית, במכירה הפומבית מוצעים לרכישה מספר עותקים זהים של מוצר, והלקוח מעוניין לקנות כמה שיותר פריטים.

```
class Agent(object):
    def __init__(self, name, budget):
        # מאתחל לקוח עם תקציב, שלא זכה בשום מוצר
    def get_budget(self):
        # מחזיר את התקציב
    def win_product(self):
        # מזכה את הלקוח בעותק נוסף של המוצר. אין שינוי לתקציב
    def count_products(self):
        # מחזיר את מספר המוצרים שהלקוח זכה בהם
    def next_bid(self):
        # מחזיר את הסכום שהלקוח יכול לשלם לכל עותק שבידיו בהנחה שיזכה
        #בעוד אחד, כלומר זה התקציב חלקי (מספר העותקים שהלקוח מחזיק + 1)
```

א. (5 נקודות)
ממשו פונקציה `total_budget` המקבלת רשימת לקוחות (נתונה כ-list של אובייקטים מטיפוס Agent), ומחזירה את סכום התקציבים שבידיהם.

```
def total_budget(agentlist):
```

ב. (5 נקודות)
ממשו פונקציה `next_winner` המקבלת רשימת לקוחות, ומחזירה את הלקוח שיכול לשלם את הסכום הגבוה ביותר לעותק, בהנחה שיזכה בעותק נוסף (במקרה של תיקו, ניתן לבחור שרירותית באחד מהסוכנים המוכנים לשלם סכום מקסימלי).

```
def next_winner(agentlist):
```

ג. (8 נקודות)
ממשו פונקציה `allocate_all` המקבלת רשימת לקוחות ומספר העותקים למכירה, ומעניקה את העותקים ללקוחות (תמורת תשלום) בכל שלב לפי הסכום הגבוה המוצע.

```
def allocate_all(agentlist, count):
```

ד. (15 נקודות)
לפעמים, קבוצה של שניים או יותר לקוחות מתאגדים במטרה לזכות ביותר עותקים ביחד.
ממשו מחלקה בשם `SuperAgent` המייצגת קבוצה של לקוחות שונים במכירה פומבית. הקבוצה צריכה:
(1) לעמוד בדרישות מהמחלקה `Agent`, (כלומר לממש את כל המתודות של המחלקה `Agent`, כך שתוכל לתפקד כלקוח רגיל בסעיפים א-ג.)
(2) הבנאי (constructor) של המחלקה SuperAgent צריך לקבל רשימה של הלקוחות המשתפים פעולה בקבוצה.
(3) חלוקת המוצרים הפנימית בין הסוכנים בקבוצה היא לפי אותו האלגוריתם בו מחלקים את המוצרים בחלוקה הראשונה (עבור מספר המוצרים בה זוכה הקבוצה).

רשימת פונקציות ומתודות מהספריה של פייתון:

ברשימה זו המילים: *list, tuple, str, dict, iterator, iterable, object, number, int, float, bool* בכתב נטוי מייצגים אובייקט כלשהו מטיפוס כזה.
הסימון *x* מייצג פרמטר שיכול להיות מטיפוסים שונים, אבל לא אובייקט כללי.
הסימונים *value, copy, string* מייצגים פלט שטיפוסו תלוי בפרמטרים שניתנים לפונקציה.
הסימון *func* מייצג פרמטר שהוא פונקציה (מספר הפרמטרים שלה והערך שהיא מחזירה תלוי בהקשר).

המרות ויצירת אובייקטים:

bool = **bool**(*x*)	convert a value to a boolean
float = **float**(*x*)	convert a number or a string to float
int = **int**(*x*)	convert a number or a string to int
list = **list**(*iterable*)	create a list of the items of **iter**(*iterable*)
str = **str**(*object*)	convert to a string, using *object*.__str__()
tuple = **tuple**(*iterable*)	create a tuple of the items of **iter**(*iterable*)
copy = **copy.deepcopy**(*object*)	create a deep copy of *object*

פונקציות מתמטיות:

number = **abs**(*number*)	return the absolute value of number
value = **max**(*iterable*)	return the maximum value in **iter**(*iterable*)
value = **max**(*x1, x2, ...*)	same, given an explicit list of values
value = **min**(*iterable*)	return the minimum value in **iter**(*iterable*)
value = **min**(*x1, x2, ...*)	same, given an explicit list of values
int = **round**(*number*)	round the number
value = **sum**(*iterable*)	return the sum of the items of **iter**(*iterable*)

איטרטורים:

iterator = **iter**(*iterable*)	create the iterator of *iterable*
value = **next**(*iterator*)	return the next value from an iterator
list = **sorted**(*iterable*)	return a sorted list of the items of **iter**(*iterable*)

מתודות של list:

list.**append**(*x*)	add *x* to the end of *list*
value = *list*.**pop**()	return and remove the last item in *list*
value = *list*.**pop**(*int*)	return and remove the item in *list*[*int*]
list.**sort**()	sort a list of comparable objects in place
list.**insert**(*int, x*)	insert an item *x* at a given position *int*

מתודות של str:

string = *str*.**join**(*iterable*)	concatenate the items of iter(*iterable*), separated by *str*
list = *str*.**split**()	split *str* into words separated by white space
list = *str*.**split**(*sep*)	split *str* into words separated by the input string *sep*